

What is a Session?

Definition: A session is a logical conversation between a web server and a web browser that spans multiple requests and responses.

- Purpose: To maintain stateful information about a user or client across multiple interactions.
- Example: Storing user login details, shopping cart contents, or user preferences.

Introduction to Session Management

Session management is crucial for maintaining user state and data across multiple requests in a web application.

Key Aspects:

- User Authentication
- State Preservation
- Security Handling

Why Use Session Management?

- Maintain user login status across pages
- Track user activities
- Improve security by managing sessions effectively

Session Management Techniques

- Using Cookies
- Using Session IDs stored on the server
- Token-based Authentication (JWT)

Session Management in FastAPI

Popular options / libraries (FastAPI is built on Starlette):

- `starlette.middleware.sessions.SessionMiddleware`: built-in signed-cookie session support (cookie-based sessions).

Redis-backed “server-side sessions” (common approach):

- Store session data in Redis (or DB), keep only a `session_id` in the cookie.
- Use `redis` / `redis.asyncio` (`redis-py`) or other backends.

Authentication ecosystems (often used alongside sessions):

- `fastapi-users` (auth flows), OAuth2 utilities, etc.

Key concepts:

- Session ID: a unique identifier assigned to each session (server-side session pattern).
- Session store: where session data is stored (Redis/MongoDB/SQL/etc.).

2. Route Parameters

FastAPI uses type hints to capture and validate URL segments.

FastAPI: path: `"/users/{user_id}"`.

Python

```
@app.get("/users/{user_id}/books/{book_id}")  
async def read_user_book(user_id: int, book_id: int):  
    return {"user_id": user_id, "book_id": book_id}
```

Middleware in FastAPI

Middleware functions sit between the request and the response. They can execute code, modify objects, and end the cycle.

Common tasks include:

- Logging: Tracking incoming requests.
- Authentication: Verifying credentials.
- Data Parsing: Handling JSON or form data.

Python

```
@app.middleware("http")
```

```
async def add_process_time_header(request, call_next):
```

```
    # Code executed before the request
```

```
    response = await call_next(request)
```

```
    # Code executed after the request
```

```
    return response
```

Starlette: Lightweight ASGI Web Framework

What it is:

- A lightweight ASGI web framework/toolkit for Python.

What it's for:

- Building fast web apps and APIs with async support.
- (WebSockets, background tasks, streaming responses, etc.)

ASGI vs WSGI:

- ASGI: Modern interface, supports async & WebSockets.
- WSGI: Older, mostly request/response only.

How it relates to FastAPI:

- FastAPI is built on top of Starlette.
- FastAPI uses Starlette for the web framework pieces (routing, middleware,
- requests/responses, WebSockets).
- FastAPI adds Pydantic-based validation and automatic OpenAPI docs.

Implementing Session Management with FastAPI

Install packages (cookie sessions + server):

```
pip install fastapi uvicorn
```

Sample Code (cookie-based session using Starlette SessionMiddleware):

Databases (Redis, MongoDB, etc.):

```
from fastapi import FastAPI °
from starlette.middleware.sessions import SessionMiddleware
app = FastAPI()

app.add_middleware(
    SessionMiddleware,
    secret_key="your-secret-key", # Important for security
    session_cookie="session",
    max_age=60 * 60, # 1 hour (example)
    same_site="lax",
    https_only=True, # set True in production behind HTTPS
)
```

What SessionMiddleware does

- It creates a session dictionary attached to every request: / secret_key:
- It stores that session data in a
- “Signed” means the cookie can’t be modified without detection (tamper-proof)
- It is not encrypted, so don't store secrets. production).

Route: /login



Code Example

```
@app.get("/login")
def login(request: Request):
    request.session["user"] =
        {"id": 123, "name": "Alice"}
    return {"message": "logged
in", "session":
request.session}
```

What happens here

- Client calls: GET /login
- The server sets
request.session['user'] = {...}
- SessionMiddleware** detects the session changed
- It **serializes** the session dict and sends it back to the browser as a Set-Cookie header
- So after calling /login, the browser stores a cookie named session that contains the signed session data.

Route: /me



Code Example

```
@app.get("/me")
def me(request: Request):
    user = request.session.get(
        "user")
    if not user:
        return JSONResponse({
            "error": "not logged in"},
            status_code=401)
    return {"user": user}
```

What happens here

- Client calls: GET /me
- Browser automatically sends the session cookie (if it has it)
- SessionMiddleware** reads and verifies the signature
- It populates `request.session` with the stored session data
- You check if 'user' exists:
 - If missing → return 401 Unauthorized
 - If present → return the user data.

Route: /logout



Code Example

```
@app.get("/logout")
def logout(request: Request):
    request.session.clear()
    return {"message": "logged
out"}
```

What happens here

- ⚙️ `request.session.clear()` empties the session dictionary.
- 🚫 Middleware then writes a new cookie (empty session) so the browser “forgets” the logged-in user.

Working Example

Cookie-session example that increments a counter (similar to req.session.views):

```
from fastapi import FastAPI, Request
from starlette.middleware.sessions import SessionMiddleware
from starlette.responses import PlainTextResponse

app = FastAPI()
app.add_middleware(SessionMiddleware, secret_key="your-secret-key")

@app.get("/")
def home(request: Request):
    if "views" in request.session:
        request.session["views"] += 1
        return PlainTextResponse(f"Views: {request.session['views']}")
    else:
        request.session["views"] = 1
        return PlainTextResponse("Welcome to the session demo. Refresh!")
```

Installing FastAPI

1. Create a directory myapp:

```
mkdir myapp; cd myapp
```

2. Create a virtual environment:

```
python -m venv .venv
```

3. Activate virtual environment:

macOS/Linux: `source .venv/bin/activate`

Windows: `.venv\Scripts\activate`

4. Install FastAPI + ASGI server:

```
pip install fastapi uvicorn[standard]
```

NOTE: Common entry point is `main.py` (similar idea to choosing `app.js` as entry point).

Python Web Application with FastAPI

In myapp directory, create a file named main.py and add the following code:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return "Hello World!"
```

Run the server:

```
uvicorn main:app --reload --port 3000
```

Then load <http://localhost:3000/> in a browser to see the output.

Web application – GET form (server)

FastAPI version (serve index.html, process GET query params):

```
from fastapi import FastAPI, Query
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def index(): # To display index.html
    return FileResponse("index.html")

@app.get("/process_get") # To process GET method
def process_get(
    fname: str = Query(...),
    lname: str = Query(...),
):
    response = {"fname": fname, "lname": lname} # preparing output in JSON format
    print(response)
    return response
```

Web application – POST form (server)

FastAPI version (process application/x-www-form-urlencoded POST body):

```
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/process_post")
def process_post(
    fname: str = Form(...),
    lname: str = Form(...),
):
    response = {"fname": fname, "lname": lname} # preparing output in JSON format
    print(response)
    return response
```

Run (example port similar to the Node slide):

```
uvicorn main:app --reload --port 8081
```

Middleware in FastAPI (like `app.use()`)

Middleware in FastAPI/Starlette is used to mount middleware functions.

Middleware can be used in several ways:

- Global (no path): When added globally, it executes for every request.
- With a “path scope” concept: FastAPI doesn’t mount middleware per-path in the same `app.use("/path", mw)` style.
- Equivalent patterns: Use APIRouter dependencies for a router prefix (runs for routes under that prefix). Use middleware but conditionally apply logic based on `request.url.path`.
- Multiple middleware functions: Multiple middlewares run in the order they are added (outer to inner).

Common tasks:

- Logging
- Authentication
- Authorization
- Data parsing (FastAPI handles JSON/form parsing automatically; middleware used for special cases)
- Serving static files
- Error handling

Example middleware:

```
import time
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    duration_ms = (time.time() - start) * 1000
    print(f"{request.method} {request.url.path} {duration_ms:.2f}ms")
    return response
```

Basic Routing

Routing refers to determining how an application responds to a client request to a particular endpoint (a URI/path) and a specific HTTP request method (GET, POST, and so on).

- Each route can have one or more handler functions (endpoint + dependencies/middleware).

Route definition structure (FastAPI):

@app.METHOD(PATH) above a Python function

Example concept:

@app.get("/items") > GET /items

@app.post("/items") > POST /items

Route methods

A route method is derived from one of the HTTP methods, and is attached to the FastAPI app/router via decorators.

Example GET and POST methods to the root of the app:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def get_home():
    return "Get request to the homepage"
@app.post("/")
def post_home() :
    return "Post request to the homepage"
```

Route paths based on strings

Match requests to /:

```
@app.get("/")
```

```
def root():
```

```
    return "root"
```

Match requests to /about:

```
@app.get("/about")
```

```
def about():
```

```
    return "about"
```

Match requests to /random:

```
@app.get("/random")
```

```
def random():
```

```
    return "random"
```

Route Parameters

Route parameters are named URL segments used to capture values at their position in the URL. The captured values are available as function parameters in FastAPI.

Example Breakdown

Route path: `/users/{userId}/books/{bookId}`

Request URL: `http://localhost:3000/users/34/books/8989`

Captured params: `userId=34, bookId=8989`

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/users/{userId}/books/{bookId}")
def read_user_book(userId: int, bookId: int):
    return {"userId": userId, "bookId": bookId}
```

FastAPI Routers (like Express Routers)

- An APIRouter is an isolated instance of routes (a “mini-application”).
- Routers can be mounted onto the main app (similar to app .use() in Express)

```
from fastapi import FastAPI, APIRouter
```

```
app = FastAPI()
```

```
# Create a router object
```

```
router = APIRouter()
```

```
# Define routes on the router
```

```
@router.get("/ping")
```

```
def ping():
```

```
    return {"msg": "pong"}
```

```
# Mount the router onto the main app
```

```
app.include_router(router)
```

Route handlers

You can provide multiple callables that behave like “middleware-style”

handlers:

- Dependencies (Depends(...)) that run before the endpoint
- Middleware that runs around the endpoint

“Handlers” can be:

- A single endpoint function
- An endpoint plus a list of dependencies
- Middleware + dependencies + endpoint combinations

Route handlers samples

A single handler can handle a route:

```
from fastapi import FastAPI  
app = FastAPI()
```

```
@app.get("/example/a")  
def example_a():  
    return "Hello from A!"
```

More than one “step” can handle a route (FastAPI uses dependencies):

```
from fastapi import FastAPI, Depends  
app = FastAPI()  
  
def log_step():  
    print("the response will be sent by the next function ...")  
  
@app.get("/example/b", dependencies=[Depends(log_step)])  
def example_b():  
    return "Hello from B!"
```

Creating a RESTful service

Steps

1. Define the domain and data
2. Organize the data in to groups
3. Create URI to resource mapping
4. Define the representations to the client
(XML, HTML, CSS, ...)
5. Link data across resources (connectedness or hypermedia)
6. Create use cases to map events/usage
7. Plan for things going wrong

Top



Down

CATALOG PAGE

☐	=====
☐	=====
☐	=====
⋮	⋮

SELECT
PRODUCT

CART PAGE

Qty	Desc	Price	Total
☐	=====	\$-	\$-
☐	=====	-	-

RECALC.

CONTINUE
SHOPPING

CHECKOUT

Maybe show
cart summary?

CHECKOUT?

Order Summary (1 line per item)

Name:

Address:

Payment:

PAY

Product:

- name
- description
- image
- price

Seller Details:

- login name
- password

Cart:

• ?

Order:

- buyer details
- payment details
- shipping status

Line Item:

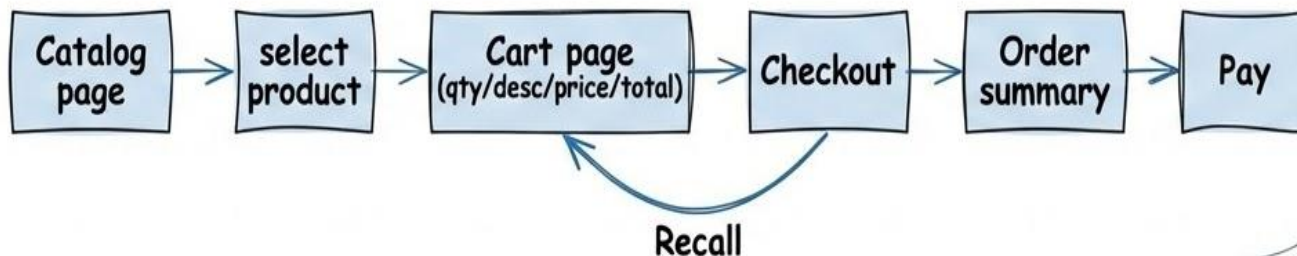
- product
- quantity
- price

a..n

1..n

REST design sketch (FastAPI interpretation)

Pages / flow:



Resources shown:



FastAPI endpoint mapping example (typical):

GET /products (catalog)

GET /products/{product_id}

GET /cart / POST /cart/items / PATCH /cart/items/{item_id} / DELETE /cart/items/{item_id}

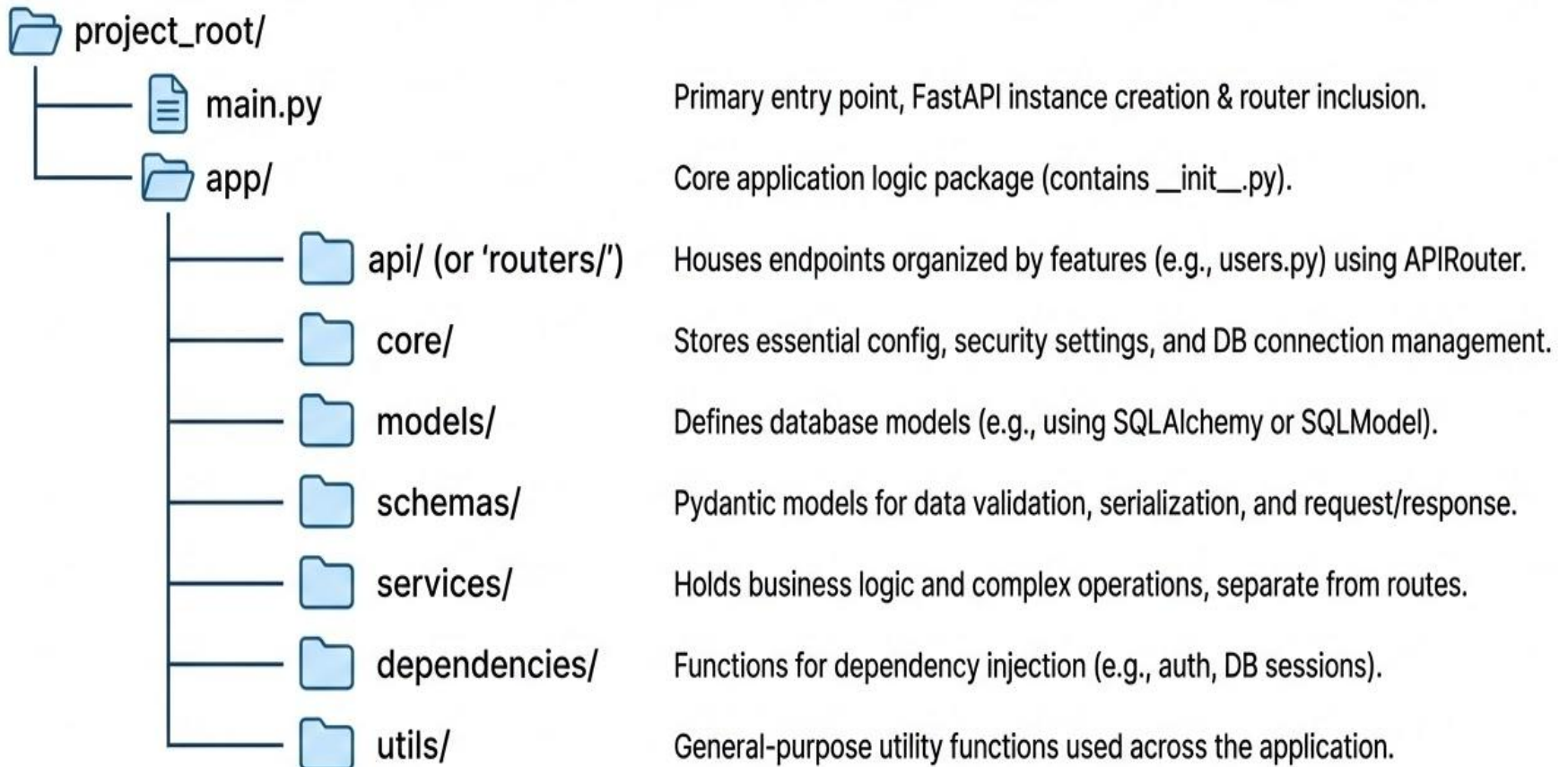
POST /checkout

GET /orders/{order_id}

POST /payments or POST /orders/{order_id}/pay

FastAPI: Scalable Project Structure & Organization

FastAPI is flexible, allowing single-file apps. However, for scalable projects, a modular structure using **APIRouter** is highly recommended to organize code by functionality.



Questions?

LLM Introduction

What is an LLM?

- A probabilistic text model that predicts the next token
- Trained on large dataset makes it better at learning statistical patterns & world knowledge
- **Transformer** is the dominant architecture
- Typical capabilities: Q&A, summarization, extraction, classification, coding
- **Examples:** GPT-class models, Llama, Gemma, Mistral, etc.

What is Tokenization and Embeddings

Tokenization = split text into model-friendly pieces (tokens) and map them to IDs.

- Why: models work on integers, not raw text; subwords (e.g., BPE) avoid OOV.
- Example (BPE): “Transformers are amazing!” → tokens like
[`"Transform"`, `"ers"`, `" are"`, `" amazing"`, `"!"`]
- **Embeddings = look up (or compute) a dense vector for each token/word/sentence.**
- Why: vectors let us measure meaning/similarity with math (dot product / cosine).
- Example (token embeddings): each token ID indexes an embedding table →
[`"Transform"`, `"ers"`, `" are"`, `" amazing"`, `"!"`] → five vectors like [`0.12`, `-0.08`, `...`, `0.31`] (say 768-D).
The model's first layer turns the ID sequence into this matrix of vectors; downstream layers use them for attention.

Tokenization & BPE (Byte Pair Encoding)

```
# pip install tiktoken
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
text = "Transformers are amazing!"
ids = enc.encode(text)
print(ids)
print(enc.decode(ids))
```

Output (example):

```
[16984, 481, 257, 14296, 0]
```

```
Transformers are amazing!
```

Key idea: **subword units** reduce OOV issues and keep vocab compact.

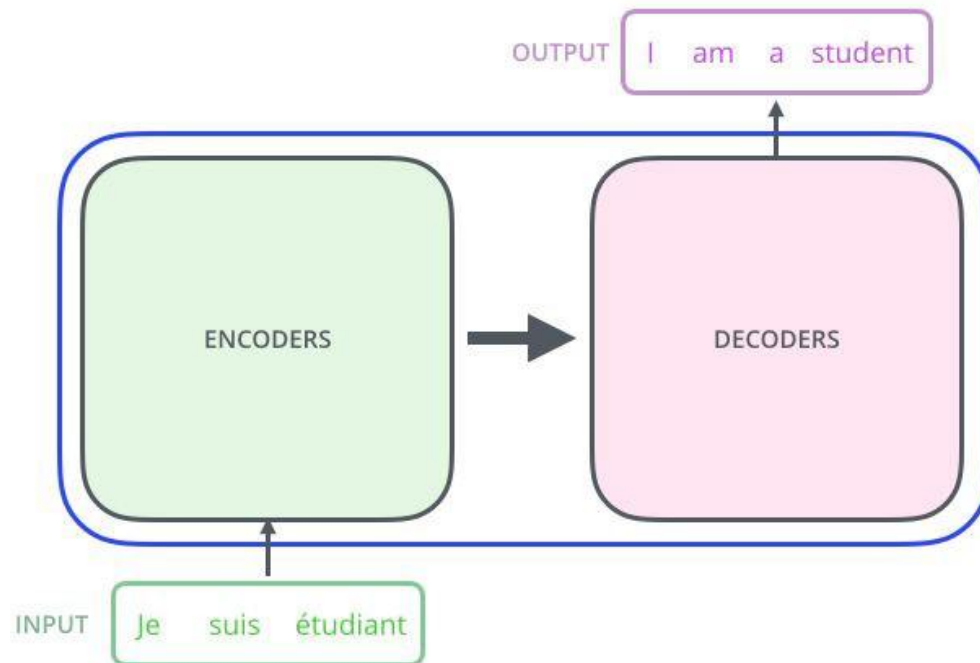
OOV = out-of-vocabulary—a word your tokenizer/model hasn't seen in its fixed vocabulary (classic word-level systems map it to an <unk> token).

Transformers at a glance



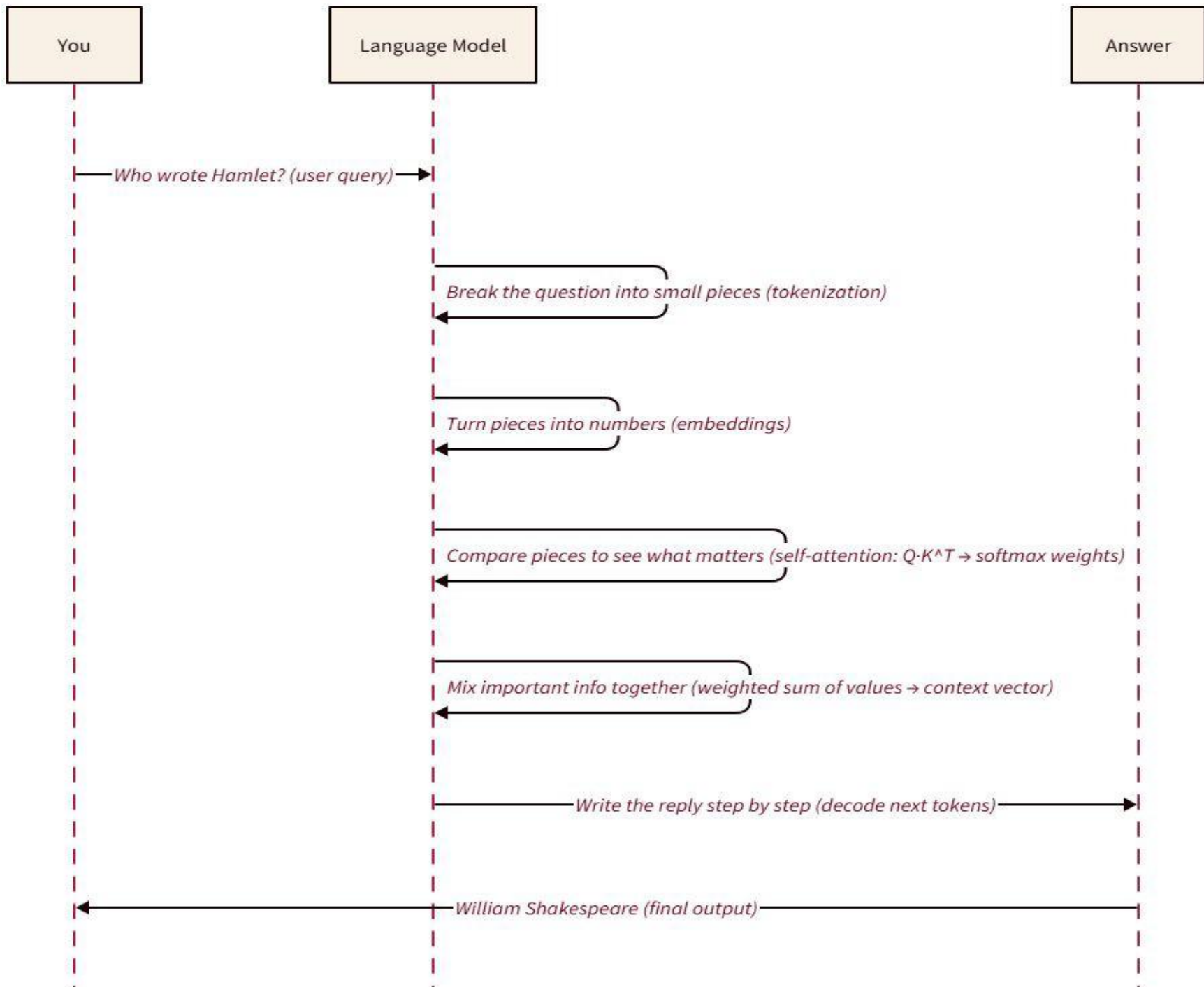
- End-to-end attention; no recurrence or convolutions
Highly parallelizable → faster training & inference

Self-attention intuition



Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$



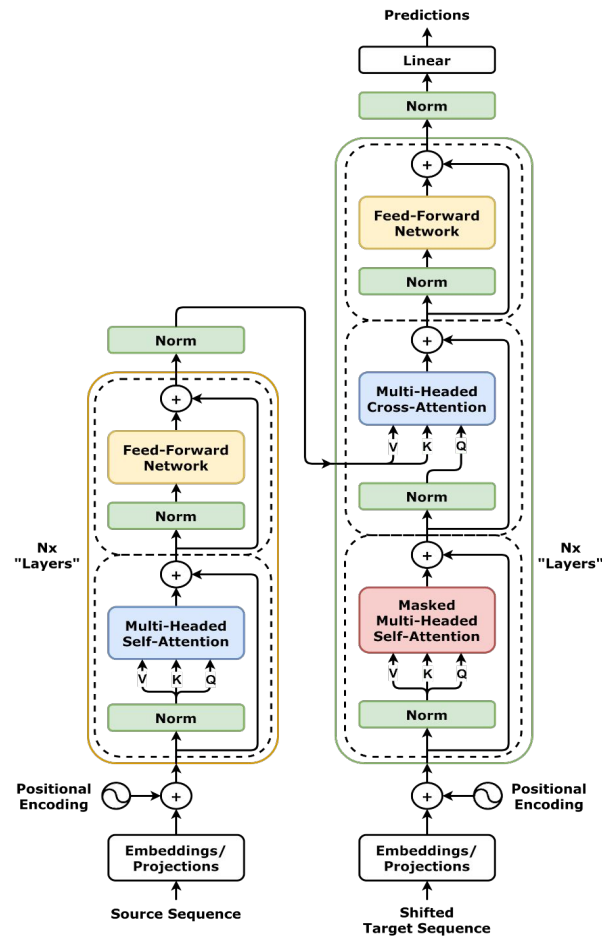
Multi-head attention

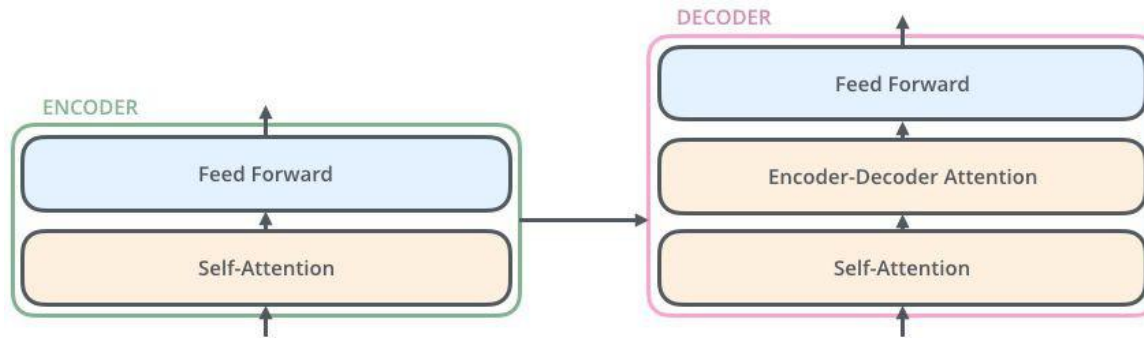
Multiple attention heads = multiple representational subspaces

Heads attend to different patterns (syntax, long-range deps, positions)

[Head 1]	attends to	nouns
[Head 2]	attends to	positional cues
[Head 3]	attends to	verb-object relations

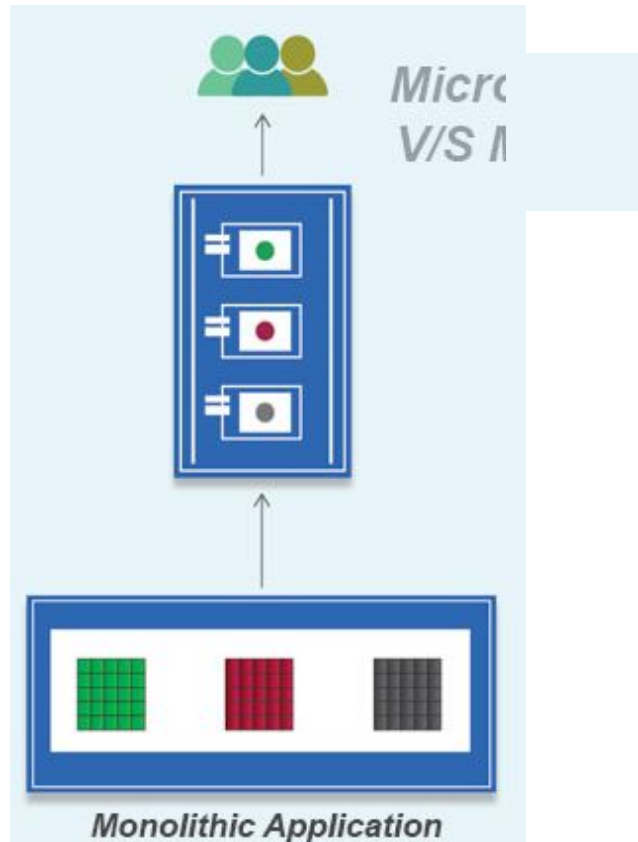
Encoder vs Decoder vs Encoder-Decoder



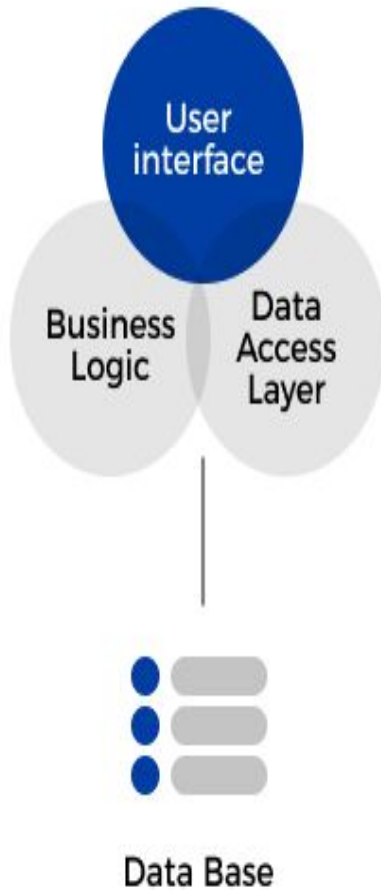


- Encoder-only = reads everything at once (bidirectional) → great for understanding.
- Decoder-only = writes one token at a time (causal) → great for generating.
- Encoder-Decoder = reads with an encoder, then writes with a decoder that can look back at what it read → great for sequence-to-sequence (translate, summarize).
- **Encoder-only** (e.g., BERT): representations for classification/extraction
- **Decoder-only** (e.g., GPT): left-to-right generation
- **Encoder-Decoder** (e.g., T5): sequence-to-sequence

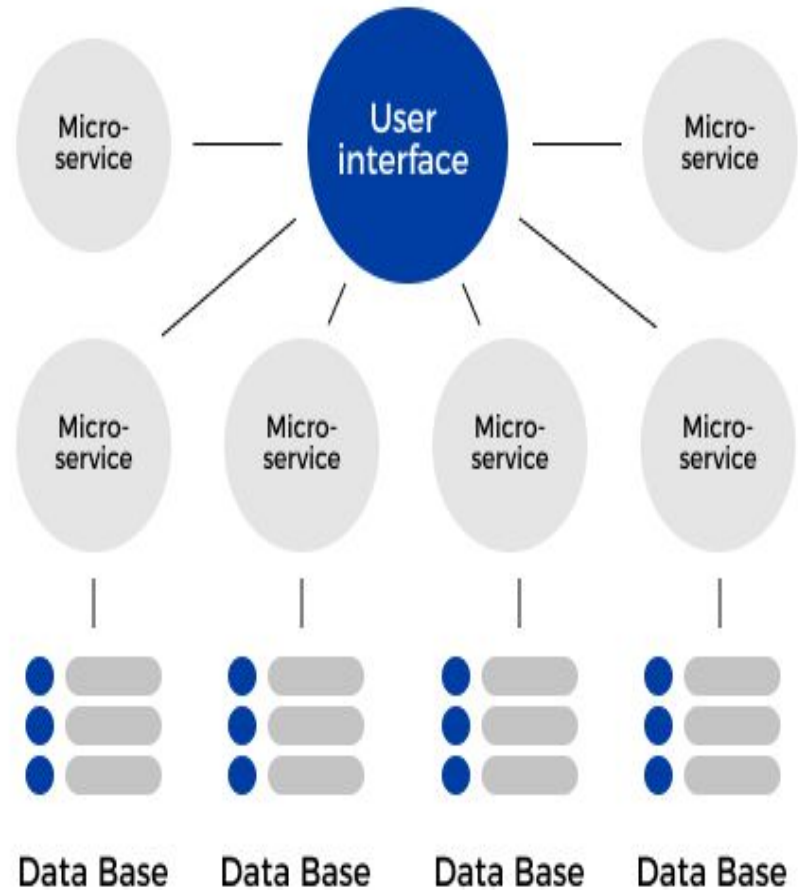
Monolithic Application



MONOLITHIC ARCHITECTURE



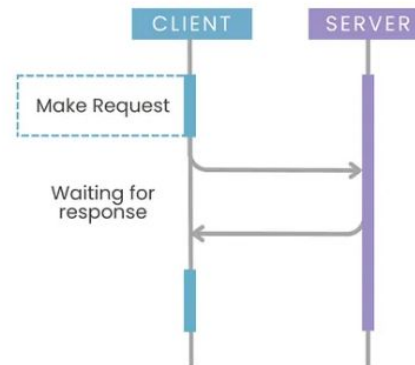
MICROSERVICE ARCHITECTURE



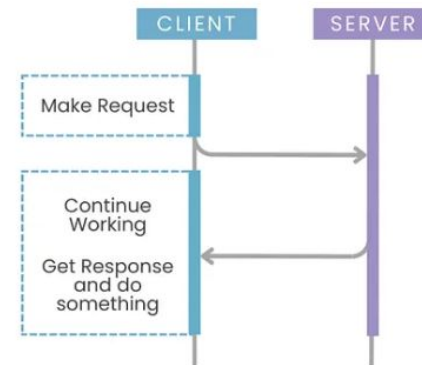
Ideal for Microservices Architecture

- Lightweight and modular
- Async support for concurrency
- Easy to containerize and deploy
- Works well with CI/CD pipelines

Synchronous

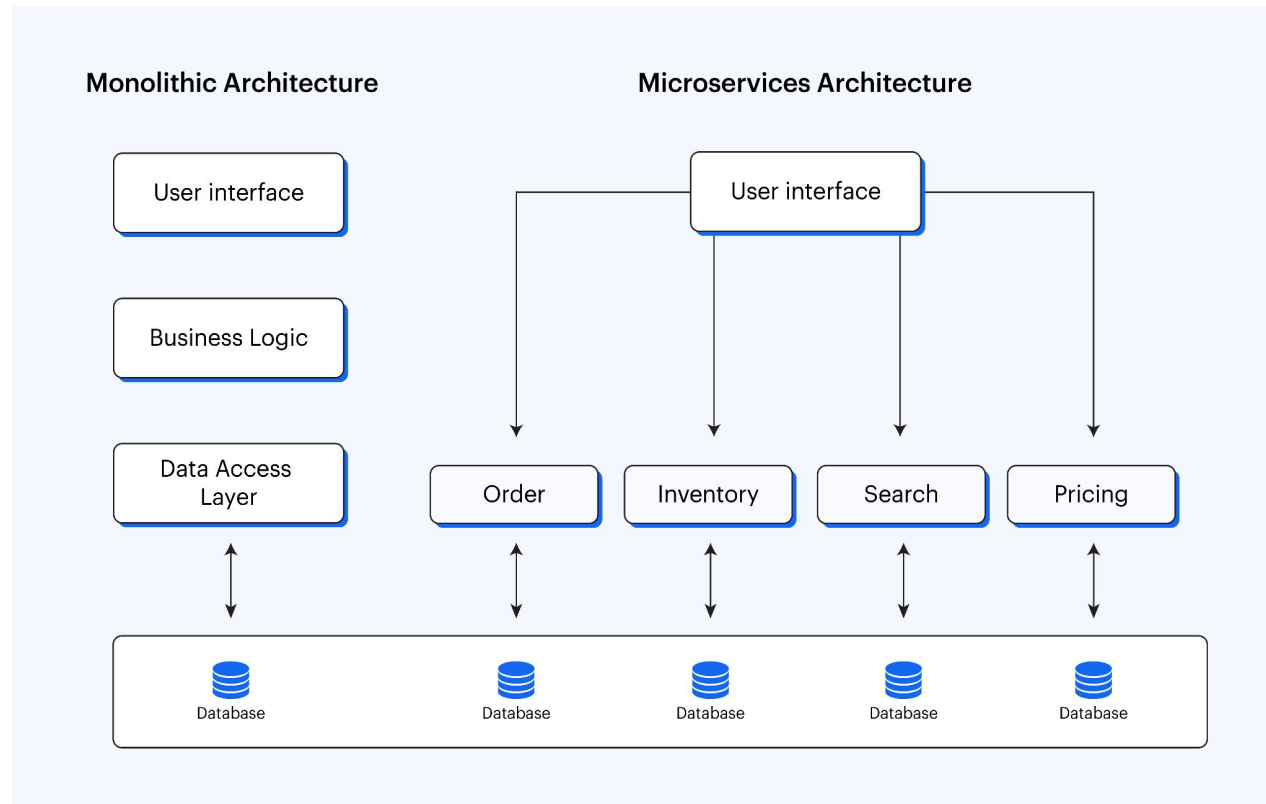


Asynchronous



Microservices

- **Microservice Independence:** Every microservice needs its codebase and database, building loosely coupled services that can be updated without relying on the other components.
- **Domain-Driven Design:** By leveraging Domain-Driven Design principles, you can be sure that every microservice closely aligns with specific business domains, minimising unnecessary inter-service communication and encouraging cohesion within every service



Why FastAPI + Agents

What you gain

Async-first (ASGI) → concurrent tool calls & non-blocking I/O.

Typed contracts with Pydantic → strict input/output for tools, safer prompts.

Auto OpenAPI docs (Swagger UI / ReDoc) for quick testing & sharing.

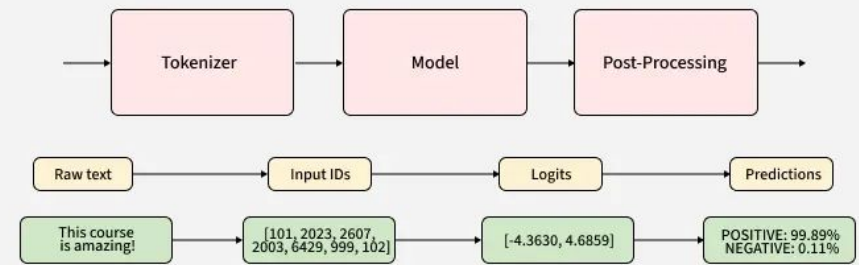
WebSockets/streaming → token-by-token updates to your UI.

FastAPI vs http.server & low-level HTTP

Why not http.server for agents?

- Not recommended for production; only basic security checks.
- No request validation or type-safe models; you hand-roll JSON parsing/errors.
- No automatic API docs / schema → harder to test tools.
- No built-in WebSockets, DI, or background tasks.

Serve a Hugging Face model



```
pip install transformers torch
```

```
from transformers import pipeline
gen = pipeline("text-generation",
model="gpt2") @app.post("/generate")

async def generate(prompt: str):
    out = gen(prompt, max_length=40,
num_return_sequences=1)[0]["generated_text"] return {"text": out}
```

*request to try
yourself:*

```
curl -X POST localhost:8000/generate -H 'Content-Type: application/json' -d '{"hello"'
```

Streaming responses (SSE)

Server

```
.from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio
app = FastAPI()
# Async generator that yields Server-Sent Event (SSE) chunks
async def stream_tokens(prompt: str):
    for token in ["This ", "is ", "streamed ", "text."]:
        # Each SSE message must end with a blank line (\n\n)
        yield f"data: {token}\n\n"
        await asyncio.sleep(0.2)

@app.get("/stream")
async def stream(prompt: str):
    return StreamingResponse(stream_tokens(prompt), media_type="text/event-stream" )
```

Client (browser):

```
<script>  
const es = new EventSource('/stream?prompt=hi');  
es.onmessage = (e) => console.log(e.data);  
</script>
```

Output

```
{"token": "This "  
{"token": "is "  
{"token": "streamed "  
{"token": "text."}
```

What is an Agent?

An agent is a system that uses an LLM to choose and execute actions (tools) in order to achieve a goal. Tools can include web search, database queries, code execution, APIs, and more.

A common approach is the ReAct pattern, which interleaves reasoning and acting in a loop:

Thought → Action (*tool = search, input = “...”*)

→ Observation (*tool result*)

→ (*repeat as needed*)

→ Final Answer

Mini example (toy)

User goal: “Find the count of ‘refund’ tickets last week and give a one-line insight.”

- **Thought:** Need last week’s tickets; then filter by “refund”; then summarize.
- **Action:** `tool=sql_query, input="SELECT created_at, title FROM tickets WHERE created_at BETWEEN '2025-09-15' AND '2025-09-21';"`
- **Observation:** 3,412 rows returned.
- **Thought:** Count titles containing “refund”.
- **Action:** `tool=code_exec, input="count = sum('refund' in t.lower() for t in titles); print(count)"`
- **Observation:** 987
- **Final Answer:** “987 refund-related tickets last week; spike followed a pricing change—consider adding clearer refund terms on checkout.”

Tool calling basics (schema)

Define tools (name, description, JSON schema for args)

Let the model select & call tools; your server executes them

Return tool **observations** to the model for next step

{

```
"type": "function",
"function": {
  "name": "search_web",
  "description": "Search the web and return a snippet",
  "parameters": {
    "type": "object",
    "properties": {"query": {"type": "string"}},
    "required": ["query"]
  }
}
}
```

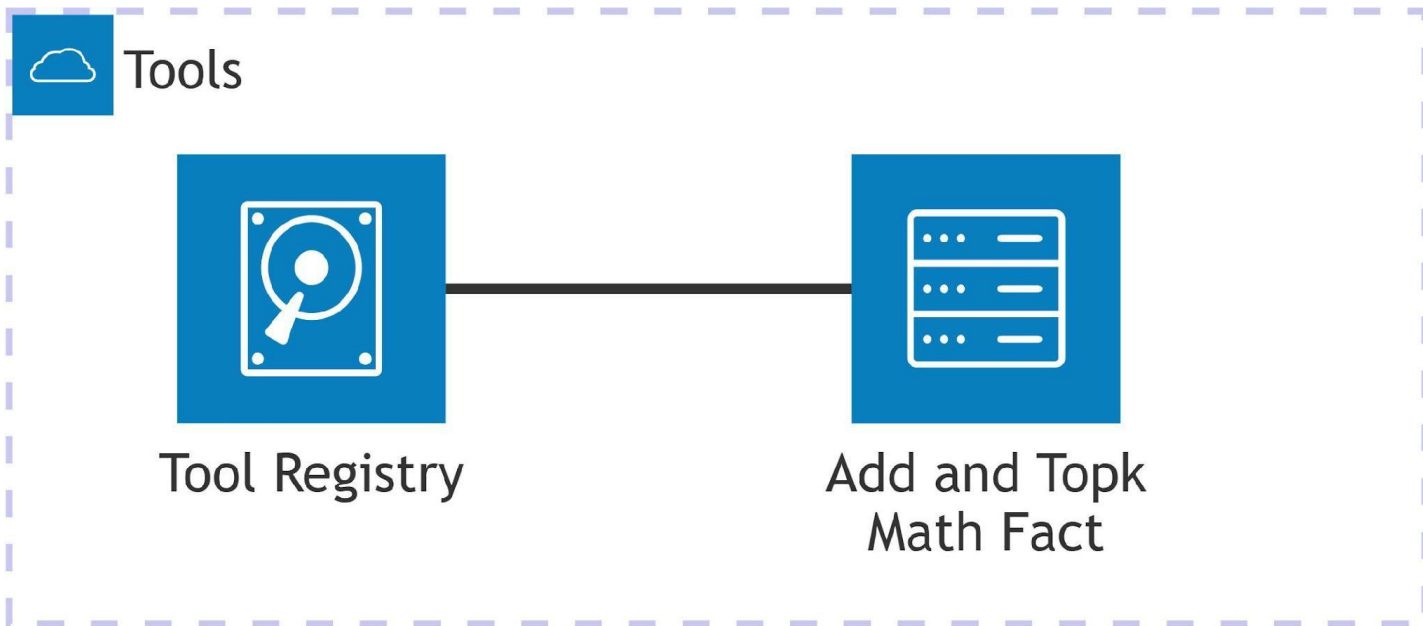
Build a tiny Agent (tools)

```
from langchain_core.tools import tool

@tool
def add(x: int, y: int) -> int: """Add two integers.""" return x + y

@tool
def topk_math_fact(topic: str) -> str: """Return a fake 'fact' for demo."""
    return f"Fun fact about {topic}: 42"

TOOLS = [add, topk_math_fact]
```



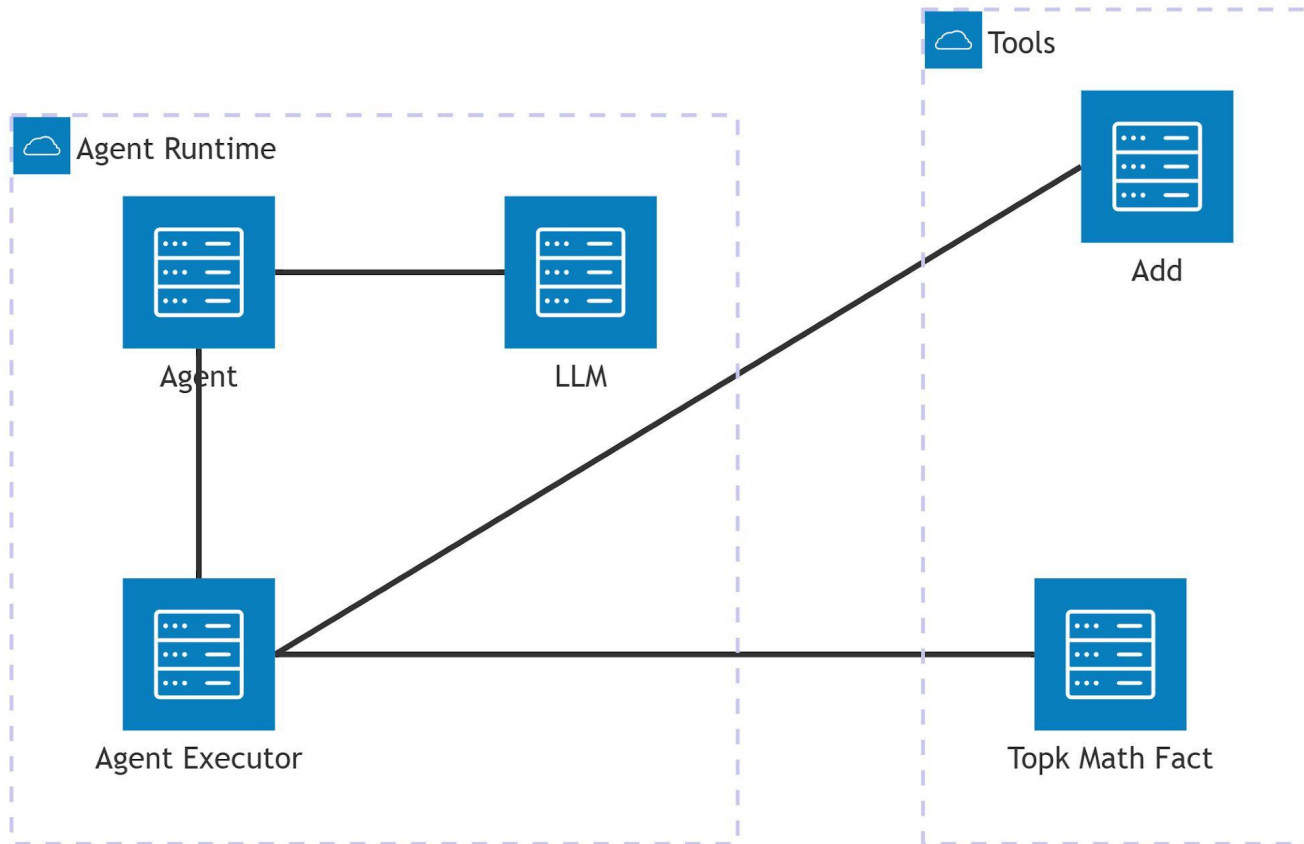
Create the Agent executor

```
llm = ChatOpenAI(model="gpt-4o-mini",
temperature=0) prompt =
ChatPromptTemplate.from_messages([
    ("system", "You are a helpful
assistant."), ("user", "{input}"),
    MessagesPlaceholder("agent_scratchpad"),
])
llm_with_tools = llm.bind_tools(TOOLS)

agent = (
    {
        "input": lambda x: x["input"],
        "agent_scratchpad": lambda x:
            format_to_openai_tool_messages(x["intermediate_steps"]),
    } | prompt | llm_with_tools | OpenAIToolsAgentOutputParser()
)

agent_executor = AgentExecutor(agent=agent, tools=TOOLS, verbose=True)
```

```
agent_executor.invoke({"input": "Add 2 and 3, then give me a fun math fact."})
```



Wrap the Agent in FastAPI

```
from pydantic import BaseModel

class AgentRequest(BaseModel): input: str

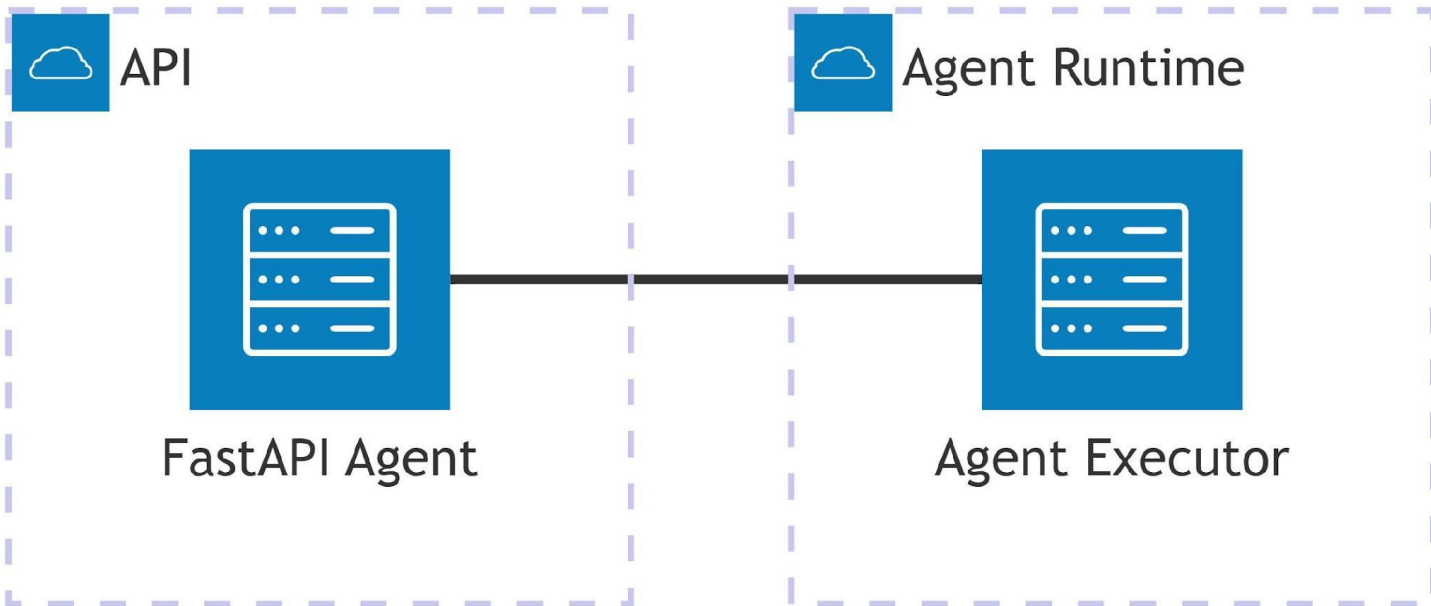
@app.post("/agent")
async def run_agent(req: AgentRequest):
    result = await agent_executor.ainvoke({"input": req.input})
    return {"output": result["output"]}
```

Request → { "input": "What is 2+3? Use add." }

Response

→

```
{"output": "2 + 3 = 5"}
```



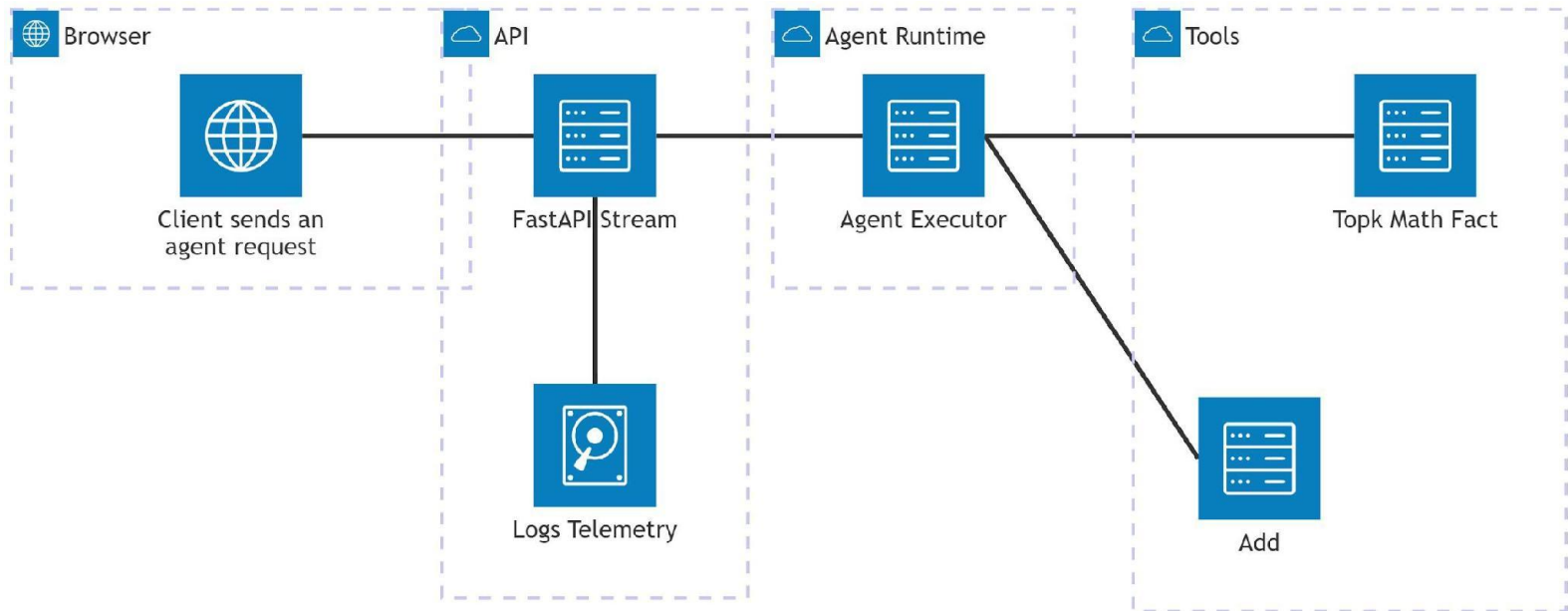
Stream the Agent (SSE)

```
from typing import AsyncIterable

async def agent_events(prompt: str) -> AsyncIterable[str]:
    async for ev in agent_executor.astream_events({"input":
        prompt}): yield f"data: {ev}\n\n"

@app.get("/agent/stream")
async def agent_stream(input: str):
    return StreamingResponse(agent_events(input),
        media_type="text/event-stream")
```

Client receives intermediate tool calls & final answer incrementally.



Testing & docs

Interactive docs at [/docs](#) (Swagger UI) and [/redoc](#)

Auto-generated schemas & examples from Pydantic

Use pytest + httpx.AsyncClient for endpoint tests

```
from httpx import AsyncClient

async def test_root():
    from main import app
    async with AsyncClient(app=app, base_url="http://test") as ac:
        r = await ac.get("/")
        assert r.json()["message"] == "Hello, FastAPI"
```

Questions?